TAM-Eval: Evaluating LLMs for Automated Unit Test Maintenance

Elena Bruches[§*], Vadim Alperovich[†*], Dari Baturova[§], Roman Derunets[§], Daniil Grebenkin[§], Georgy Mkrtchyan[†]
Oleg Sedukhin[§], Mikhail Klementev[§], Ivan Bondarenko[‡], Nikolay Bushkov[†], Stanislav Moiseev[†]
[§]*Siberian Neuronets LLC*, Novosibirsk, Russia
[†]*T-Technologies*, Moscow, Russia
[‡]*Novosibirsk State University*, Novosibirsk, Russia
{bruches, baturova, derunets, grebenkin, sedukhin, klementev}@sibnn.ai
{v.alperovich, g.p.mkrtchyan, n.bushkov, s.moiseev}@t-tech.dev
i.bondarenko@g.nsu.ru
[*] Both authors contributed equally to this research.

***Abstract*—While Large Language Models (LLMs) have shown promise in software engineering, their application to unit testing remains largely confined to isolated test generation or oracle prediction, neglecting the broader challenge of test suite maintenance.**

We introduce TAM-Eval (Test Automated Maintenance Evaluation), a framework and benchmark designed to evaluate model performance across three core test maintenance scenarios: creation, repair, and updating of test suites. Unlike prior work limited to function-level tasks, TAM-Eval operates at the test file level, while maintaining access to full repository context during isolated evaluation, better reflecting real-world maintenance workflows.

Our benchmark comprises 1,539 automatically extracted and validated scenarios from Python, Java, and Go projects. TAM-Eval supports system-agnostic evaluation of LLMs, using a reference-free protocol based on test suite pass rate, code coverage, and mutation testing.

Empirical results indicate that state-of-the-art LLMs have limited capabilities in realistic test maintenance processes and yield only marginal improvements in test effectiveness. We release TAM-Eval as an open-source framework to support future research in automated software testing. Our data and code are available at https://github.com/trndcenter/TAM-Eval.

***Index Terms*—unit testing, test maintenance, LLM, benchmarks, software engineering automation, LLM4SE**

I. Introduction

Unit testing plays a pivotal role in ensuring robustness and reliability by validating individual components of software [1], with industry reports estimating that testing consumes up to 25% of total project budgets [2].

Recent advances in large language models (LLMs) have significantly impacted software engineering tasks, including code generation [3], test generation [4], documentation [5], refactoring [6], and bug fixing [7]. Despite strong performance in function-level test generation, existing LLM-based tools largely produce isolated tests or assertions and do not address the broader lifecycle of real-world test suites.

In practice, however, software quality depends heavily on test maintenance — the continuous generation, repair, and updating of tests as code evolves. Failing to maintain tests leads to outdated suites, broken pipelines, and undetected regressions. Yet, this setting remains underexplored in current research.

Technical Challenges. Evaluating automated test maintenance requires context-aware reasoning over both production code and existing tests, as well as dynamic execution environments that support correctness checking and incremental evaluation across dimensions such as execution correctness, coverage and mutation score.

Our Contributions. We present **TAM-Eval** (**T**est **A**utomated **M**aintenance **Eval**uation), a benchmark and evaluation framework for comprehensive unit test maintenance assessment. An overview of its pipeline is shown in Fig. 1 Its key contributions are as follows:

- **Test Maintenance Scenarios:** The novelty of TAM-Eval is that it addresses the complete test maintenance cycle — *generation*, *repair*, and *update* of test suites at the granularity of entire test files, reflecting realistic developer workflows.
- **Curated Real-world Dataset:** The benchmark includes 1,539 validated scenarios from actively maintained open-source repositories, which were selected via a multi-stage filtering pipeline to ensure quality with measures to minimize data contamination.
- **Unified and Feedback-Aware Evaluation:** All tasks are prompted through a unified prompt template with minimal instructions and iterative execution feedback, testing the model's ability to generalize across maintenance types and simulate practical workflows.
- **Extensible, Open Evaluation Suite:** TAM-Eval supports reference-free evaluation via test pass rate, coverage, and mutation testing. It is language-agnostic, easy to extend, and fully open-sourced to facilitate reproducibility and community benchmarking.
- **Empirical Analysis of LLMs:** We evaluate advanced SOTA (state-of-the-art) LLMs, revealing that even the best-performing models exhibit large variability across tasks and languages, with mutation coverage improve-

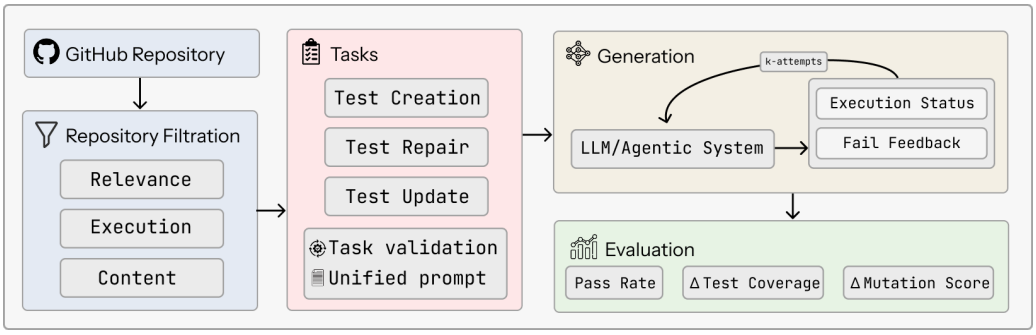

Fig. 1. Overview of TAM-Eval, a framework and benchmark for evaluating LLMs on unit test maintenance. The pipeline filters GitHub repositories, constructs tasks for test creation, repair, and update with execution-based validation, performs LLM-driven test generation with iterative feedback, and computes Pass Rate, ΔTest Coverage, and ΔMutation Score.

ments rarely exceeding 12 percentage points, indicating substantial room for advancement. This highlights the need for developing more context-aware methods and distilling supervision from verifiers such as compilers, mutation, and coverage signals.

Outline. Section II reviews relevant work. Section III presents our benchmark design. Section IV details the evaluation framework. Section V describes the experimental setup. Section VI reports our experimental results.

II. Background

A. Unit tests generation

The attempts to automate the unit testing process started in the last century [8] and focused on code coverage, such as Randoop [9] or EvoSuite [10].

With the rise of LLMs, various prompting techniques [11] were used to generate unit tests. Whereas earlier models had difficulties with self-repair and test improvement [12], [13], such methods have received a second wind with the advent of reasoning approaches [14], [15]. For example, ChatTester [16] leverages ChatGPT itself to improve the quality of its generated tests. [11] Libro [17] focuses on post-processing steps that help discern when LLMs are effective and rank the produced tests according to their validity.

Rapid development in code generation and particularly in test generation has led to the need to create high-quality assessment systems for different test-related tasks like test repairing or test updating.

B. Unit tests repairing

One of the important tasks associated with unit tests is the task of test repair, a process that helps validate the code and improve its performance.

The frameworks and methods like TestART [18], ChatUniTest [19], already mentioned ChatTester [16] and others [20], [21] use the APR (Automated Program Repair) idea, but the proposed pipelines also contain the repair stage, which can be rule-based, LLM-based, or combined. The rule-based repair strategy can include steps like syntax analysis and searching test annotations for test extraction, copying imports from focal file (tested file) to test file [19], mutations of different entities to increase test correctness and robustness [18]. However, some research shows that in spite of the fact that repair can be a part of the test generation pipeline, these tasks require a different set of LLM skills [22]. Nevertheless, the use of different LLM strategies and prompt engineering allows the creation of architectures like TestGen-LLM [23], which was fine-tuned to increase test's coverage and reliability.

C. Unit tests updating

Unit tests need to be periodically updated and maintained: they may not initially cover all the necessary functionality or be unstable to mutations, for which purpose systems are developed that can improve existing tests, such as Meta's TestGen-LLM tool [23].

The solution of this issue may be associated with using commits that modify the tests. Application of the corresponding concept can be found in the dataset with vulnerabilities of open-source Android projects, collected on the basis of commits with vulnerabilities and fix-commits [24]. The GitBug-Actions benchmark also used a buggy commit and a subsequent fixing commit. Long Code Arena [25] used commit data for CI Builds Repair tasks as well as for Project-Level Code Completion.

Noteworthy is SWE-smith [26], where to create a training set the authors took a current commit and brought its state to a tests failure. In one strategy, they reverted pull requests on commits back to the point where the tests started failing. SWT-Bench [22] extends this paradigm by explicitly pairing GitHub issues with their corresponding fix commits, requiring generated tests to fail on pre-fix code while passing on post-fix implementations.

D. Unit-testing related benchmarks

Unlike traditional code-evaluation benchmarks focused mainly on assessing code-generation abilities of large language models (LLMs), specialized benchmarks geared toward evaluating unit-testing capabilities have recently emerged. Each benchmark diverges in the specific tasks that it evaluates.

For instance, TestBench [27] concentrates on Java programs, consisting of 108 meticulously curated examples drawn

from nine large-scale open-source projects. It evaluates LLMs across five key dimensions: syntactic correctness, compilation success, test validity, code coverage, and defect-detection rates. Another notable benchmark is ProjectTests [28], which spans three major programming languages (Python, Java, and JavaScript) and incorporates 20 exemplary projects per language. These projects underwent extensive manual selection and refinement, ensuring the dataset's reliability. However, the high cost of expanding this dataset might hinder its long-term utility. SWT-Bench [22], derived from SWE-Bench [29], boasts over 2,000 Python samples, showcasing diverse application contexts. Meanwhile, CPP-UT-Bench [30] compiles 2,653 manually constructed code-unittest pairs from 14 renowned open-source C++ repositories. Notably, its cross-domain representation enhances the relevance to real-world problems. Additionally, CLOVER [31] addresses Python-specific unit test generation by treating the task as both code completion and open-ended code generation.

Some benchmarks delve into more sophisticated goals beyond simple test-case generation. For example, TESTEVAL [32] examines three distinct dimensions: overall code coverage (cov@k), targeted line and branch coverage, and path coverage. However, its absence of repository-level data restricts its practical relevance.

In addition, several benchmarks emulate real-world scenarios. DevBench [33] simulates the entire software development lifecycle (SDLC), integrating tasks such as software design, environment setup, implementation, acceptance testing, and unit testing. Similarly, TestGenEval [34] prioritizes the generation of comprehensive test suites, stressing initial test authorship, suite expansion, and coverage improvement.

In contrast with the benchmarks mentioned above, we propose a fully automated pipeline to collect high-quality data for unit test creation, test repair, and test updating tasks.

III. Benchmark Construction

We provide a benchmark for evaluation of automated unit test maintenance process, which contains 1,539 samples for 3 programming languages: Java, Python, and Go. These samples were meticulously filtered and annotated with supplementary metadata to approximate real-world scenarios. The basic information about the benchmark is presented in Table I.

Below, we describe the benchmark construction process in detail.

Stage 1. Repository Selection

We collected open-source GitHub repositories via GitHub API that satisfy quality and relevance criteria: updated after `2020`, and the focal files used in our benchmark to have their latest commit after `2025` (`2024` for Java), at least 40 stars, permissive licenses (MIT, Apache-2.0), and at least two contributors. We expect that these criteria help avoid LLM training contamination and can be updated over time.

After this stage, 94.1% of the original data was filtered out.

Stage 2. Execution-based Filtering

Since our framework executes code during evaluation, it is crucial to ensure that candidate projects are both buildable and runnable within reasonable resource and time constraints. To this end, we apply the following filters:

Automated build process. Only projects that were constructed entirely through predefined commands, without any manual intervention, were included.

Test execution. We require that the test suite runs within 30 seconds.

Test stability. To eliminate flaky or non-deterministic tests, each test is executed twice. Only those that pass consistently across both runs are retained. At this step, it became also possible to distinguish suitable focal-test file pairs through the analysis of test file imports and filenames, according to the features of every chosen programming language. The test files could contain only imports from focal file, both of the files in a suite could not have imports from non-standard packages, etc.

Test coverage level. We exclude focal-test file pairs where the original test suite achieves less than 40% line coverage of the corresponding focal function. This ensures that retained test suites are not trivially under-specified and that there is a meaningful baseline from which to assess improvements in coverage and mutation score.

After applying these filters, we retain only those repositories that contain at least **five focal-test file pairs** that satisfy the above conditions. An additional 12.8% of the data present at the beginning of the stage was filtered out.

Stage 3. Content-based Filtering

To ensure that the dataset comprises relevant and high-quality samples suitable for robust evaluation, we applied a series of content-based checks:

Minimum number of test cases. Samples containing fewer than two test cases in the associated test file were excluded.

Function complexity in focal files. Pairs where the focal file does not contain at least one function with five or more lines of executable code were removed. This excludes trivial or stub functions unlikely to provide meaningful evaluation challenges.

File size constraints. Pairs where focal or test files contain fewer than 20 lines or exceed the 99th percentile were discarded.

Comment density. Samples in which comments made up over 70% of the focal file were excluded, ensuring that the model receives primarily executable content. We believe that a high ratio of comments may cause the data leakage and help models to generate tests based on the documentation but not on the code itself.

Generated code. Automatically generated files (e.g., containing "Generated by" markers) were removed to avoid non-human-authored samples.

Finally, to prevent over-representation of large repositories and to promote diversity in the benchmark, we performed

balanced sampling across projects. Specifically, we randomly sampled up to 10 focal–test file pairs per repository.

After the third stage, 45% of the data remaining from the previous stage was filtered out.

Stage 4. Test Maintenance Tasks Creation

In the final stage, we derive concrete evaluation tasks from the filtered and executable focal–test file pairs, the samples were assigned to tasks with no overlap. These tasks are designed to reflect core subproblems within the domain of automated test maintenance.

A. Test Creation Task – generating new tests for previously untested or insufficiently tested code. We simulate three distinct test creation scenarios by applying structured modifications to the collected dataset:

From Scratch: we cleared the contents of the test file corresponding to the focal file, effectively removing the entire test suite.

Add New Tests: selecting test files with partial coverage of the focal file (i.e. coverage less than 100%), requiring extension to cover uncovered logic.

Recover Tests: removing a subset of test cases from test files with high coverage, thereby creating the need to generate missing test logic while maintaining overall coverage. The described setup guarantees the feasibility of new test generation. In contrast, lower coverage in the *Add New Test* setting does not always imply that additional test cases are necessary, useful, or even feasible.

B. Test Repair Task – correcting broken, outdated, or inconsistent tests. To construct this task samples, two sources of faulty tests were implemented: (1) synthetically injected errors via heuristics based on the `tree-sitter`[1], and (2) LLM-generated defects prompted to produce diverse error types and complexities, grounded in both human-written and synthetic test suites. Specifically, Qwen2.5 Coder 32B was used to generate broken test code.

Below we provide the description of the defects which were incorporated into the test suites.

Syntax errors (4.07% of repair challenge): basic syntactic violations such as missing braces, commas, or extraneous tokens. This category ensures that models can perform minimal syntax correction

Execution-targeted faults (47.37% of repair challenge): syntactically valid tests that fail at compile or runtime due to missing imports, undefined identifiers, invalid calls, or exceptions. Introduced via `tree-sitter`-based heuristics and LLM prompting, and validated through execution logs.

Coverage-targeted breakdowns (17.77% of repair challenge): tests that compile and run but add little or no value to code coverage or mutation score, often due to redundant logic or superficial variation.

Efficiency-targeted breakdowns (30.74% of repair challenge): tests lacking assertions or checks, thus offering weak behavioral guarantees. Simulated by removing verification logic to assess the model's ability to restore meaningful oracles.

[1]https://tree-sitter.github.io/tree-sitter/

TABLE I
BENCHMARK STATISTICS

Lang	# samples	# repos	Avg. focal LOC	Avg. test LOC	# test cases	Avg. Test-Cov (%)	Avg. Mut-Cov (%)
Python	442	55	151	89	5.59	50.40	52.27
Java	495	55	96	66	3.89	32.45	27.50
Go	602	82	119	114	3.22	31.70	43.24
Overall	1539	192	120	91	4.11	37.32	17.81

C. Test Updating Task – adapting existing tests to reflect changes in the associated production code. To simulate a realistic test maintenance process, we reverted the test file to its state k commits earlier while keeping the current focal file. This ensures that test updates are both necessary and feasible, as evidenced by degradation in test coverage, mutation score, or execution correctness.

We identify suitable commits by traversing the test file's history in reverse chronological order and selecting the first commit that modifies at least one line of executable code. A sample is retained if test or mutation coverage drops by at least 5% compared to the current version, or if the reverted test fails to execute correctly with the current focal file.

IV. EVALUATION FRAMEWORK

To assess LLM-driven systems for automated unit test maintenance, we introduce a reference-free and fully automated evaluation pipeline. The pipeline supports all three constructed tasks introduced in this work: test creation, repair, and updating.

A. Metrics

We define a set of interpretable test-oriented metrics that capture the effectiveness of generated test suites in improving coverage, detecting faults, and ensuring correctness.

PassRate. A basic yet essential metric: The ratio of successful test executions to total executed tests. It serves as a filter for utility, as failing tests cannot reliably validate correctness. Similar execution-based correctness signals have been used previously, e.g., in *MERA Code* via the *CodeCorrectness* benchmark [35].

ΔTestCov. Line-level test coverage reflects how thoroughly the test suite exercises the focal file. For a given focal-test file pair, we define:

$$\texttt{TestCov} = \frac{\text{\# lines executed by tests}}{\text{\# total executable lines in the focal file}} \times 100$$

We compute the coverage both before $\texttt{TestCov}_{\text{initial}}$ and after $\texttt{TestCov}_{\text{final}}$ modifications introduced by an evaluated system. The relative improvement is:

$$\Delta\texttt{TestCov} = \texttt{TestCov}_{\text{final}} - \texttt{TestCov}_{\text{initial}}$$

ΔMutCov. Mutation testing evaluates whether a test suite can distinguish the correct implementation from faulty variants. Let F be the original file under test. We generate mutants $F_1^{mut}, \ldots, F_k^{mut}$ by applying small code changes (e.g., negating a condition or altering an operator). An ideal test suite should: pass on F (the unmodified code), fail on as many mutants F_i^{mut} as possible ("failed" or "killed" mutants). The number of generated mutants depends on the structure of the target code.

Therefore, the formula for mutation coverage is:

$$\texttt{MutCov} = \frac{\text{\# failed mutants}}{\text{\# total mutants}} \times 100,$$

And subsequently:

$$\Delta\texttt{MutCov} = \texttt{MutCov}_{\text{final}} - \texttt{MutCov}_{\text{initial}}$$

B. Evaluation Pipeline

Each benchmark sample undergoes a reproducible and fully automated evaluation process consisting of the following steps:

Sanity Checks: The generated test file is validated for non-triviality – we discard outputs that are empty or exact duplicates of the input.

Syntax Validation: Language-specific parsers (based on `tree-sitter`) check that the generated code is syntactically valid and compilable.

Test Execution and Coverage Analysis: The test suite is executed against the corresponding focal file. We collect line-level test coverage using language-specific tools to compute ΔTestCov.

Mutation Testing: We generate a set of mutants and evaluate how many are failed by the test suite to compute ΔMutCov. For each run, the set of mutants is fixed, which means that e.g. mutpy (for Python) generates all possible mutants for the focal file without any sampling.

Metric Aggregation: Individual deltas are computed for each metric and then aggregated across the benchmark to produce average performance indicators.

C. Infrastructure

To ensure consistency, scalability, and isolation, all evaluations are conducted in sandboxed Docker environments. For each project, we automatically detect appropriate build and test commands using static heuristics based on language conventions and project structure.

We use the following tools for execution and analysis:

Coverage analysis: `coverage.py`[2] for Python, `JaCoCo`[3] for Java, and `cover package` [4] for Go.

Mutation testing: `mutpy`[5] for Python, `PIT`[6] for Java, and `go-mutesting`[7] for Go.

[2]https://github.com/nedbat/coveragepy
[3]https://github.com/jacoco/jacoco
[4]https://pkg.go.dev/cmd/cover
[5]https://github.com/mutpy/mutpy
[6]https://pitest.org
[7]https://github.com/avito-tech/go-mutesting

TABLE II
MAIN RESULTS (ATTEMPT@3)

Model	Pass Rate (%)	$\overline{\Delta}$ TestCov	$\overline{\Delta}$ MutCov
Devstral-Small [36]	11.04	$\uparrow 4.8 \pm 0.5$	$\uparrow 2.3 \pm 0.4$
Qwen3 Coder 480B A35B [37]	18.58	$\uparrow 8.6 \pm 0.6$	$\uparrow 4.8 \pm 0.5$
DeepSeek V3.1 671B [38]	10.33	$\uparrow 5.8 \pm 0.5$	$\uparrow 2.9 \pm 0.4$
GPT-OSS-120B [39]	**32.81**	$\uparrow$ **16.3** ± 0.8	$\uparrow$ **8.5** ± 0.7
Gemini 2.5 Flash [40]	9.94	$\uparrow 4.8 \pm 0.5$	$\uparrow 2.6 \pm 0.4$
GPT-5 [41]	**42.37**	$\uparrow$ **20.8** ± 1.0	$\uparrow$ **11.7** ± 0.8

Our infrastructure is modular and language-extensible, allowing straightforward addition of new languages or testing tools in future iterations of the framework.

V. EXPERIMENTAL SETUP

A. Generation stage

The model receives as input the focal file and its corresponding test file contents. While our default setup limits the input context to streamline and accelerate the evaluation process, the framework allows full-repository context use. The model is prompted to rewrite the entire test file from scratch to enhance the provided test suite by increasing test coverage and effectiveness.

Failure Recovery via Attempts. Each model is allowed up to k *attempts* to generate a valid output. If the initial generation fails, the next attempt includes failure feedback – such as syntax errors, compiler messages, or stack traces – automatically extracted with minimal preprocessing.

Unified Prompt Instruction. All test maintenance tasks – generation, repair, and update – share a single unified instruction template. The prompt provides minimal task-specific guidance, relying on the model's capability to infer intent from context.

B. Inference Configuration

We evaluate a diverse set of LLMs, both open-source and proprietary, to benchmark their performance on automated unit test maintenance. The list of models is presented in Table II.

We queried all models via the OpenRouter API[8] using the chat/completions endpoint through the AsyncOpenAI client. Sampling was performed with a fixed random seed and temperature of 0.25 to ensure reproducibility while allowing minimal stochasticity. Default system prompts were used unless stated otherwise; for Qwen3 Coder 480B A35B we enabled the `/no_think` option to suppress verbose reasoning and encourage direct edits.

VI. RESULTS

We present the evaluation results of TAM-EVAL across various model configurations and task dimensions.

[8]https://openrouter.ai

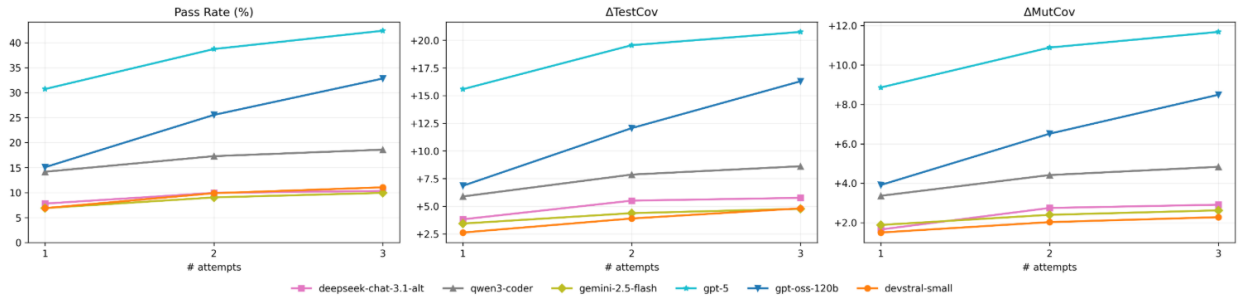

Fig. 2. Dynamics across three metrics: Pass Rate, mean Δ Test Coverage, and mean Δ Mutation Coverage by attempts

A. Main Results

Table II summarizes the overall performance of the evaluated models with a maximum of three recovery attempts (Attempt@3). GPT-5 achieves the highest $PassRate$ of 42.3%, significantly outperforming other models, highlighting its superior ability to generate effective test cases. GPT-OSS-120B follows with a $PassRate$ of 32.8%, the largest Qwen's model Qwen3 Coder 480B A35B demonstrates moderate performance as well as DeepSeek V3.1 671B.

We also analyze the impact of multiple attempts on model performance, as shown in Fig. 2. Across all models, all three metrics generally improve with additional attempts, with GPT-5 and GPT-OSS-120B showing the most consistent gains, underscoring their robustness in iterative recovery abilities and utilizing fail feedback signals.

However, current models demonstrate limited capability in test maintenance tasks on their first attempt without fail feedback context, as evidenced by the low initial performance in Fig. 2, except GPT-5, which achieves a $PassRate$ of 30.7% at Attempt@1.

B. Performance by Language

Table III presents models' performance by programming language at Attempt@3. GPT-5 consistently outperforms all other models, achieving the highest average *Pass Rate* across languages, most notably dominating Go and Java. GPT-5 also shows the largest mean improvements across languages having $\overline{\Delta}$TestCov 18.7 and $\overline{\Delta}$MutCov 10.2). Although GPT-OSS-120B is the second-best model overall, it achieves the highest Python line-level and mutation test coverage. Qwen3 Coder 480B A35B provides decent metrics, delivering particularly strong Go results.

Surprisingly, Go has emerged as the most suitable language for modern LLMs, thanks to its concise syntax, strict typing, and minimal semantic noise — making code generation more accurate, predictable, and maintainable. Java, although it sometimes gives a fairly high *Pass Rate* (e.g., 29.3% for GPT-5), shows notably lower $\overline{\Delta}$TestCov and $\overline{\Delta}$MutCov, highlighting that test executability and passing do not always translate into meaningful improvements in coverage for more verbose, statically-typed languages.

Across all evaluated languages, Qwen3 Coder 480B A35B consistently produces the shortest code snippets, whereas Gemini 2.5 Flash outputs the longest. Specifically, for Python, the mean test file lengths range from 3,698 characters (Qwen3 Coder 480B A35B) to 15,641 characters (Gemini 2.5 Flash), for Go: from 4,410 (Qwen3 Coder 480B A35B) to 13,463 (Gemini 2.5 Flash), and for Java: from 3,017 (Qwen3 Coder 480B A35B) to 12,711 (Gemini 2.5 Flash).

Interestingly, despite having shorter overall test files, Java exhibits the highest assert density. Models produce an average of 15 asserts per file (Qwen3 Coder 480B A35B) up to 66 asserts (Gemini 2.5 Flash). By contrast, Python averages between 12 (Qwen3 Coder 480B A35B) and 63 (Gemini 2.5 Flash) asserts, while Go shows significantly fewer assertions, ranging from just 10 (Qwen3 Coder 480B A35B) to 30 (Gemini 2.5 Flash).

Additionally, the distribution of test cases mirrors the assert count trends: Python typically contains between 9 (Qwen3 Coder 480B A35B) and 19 (Devstral-Small) test cases per file, Go ranges from 7 (Qwen3 Coder 480B A35B, GPT-OSS-120B, Gemini 2.5 Flash) to 11 (Devstral-Small), and Java spans from 10 (GPT-OSS-120B) to 16 (Devstral-Small).

C. Performance by Task

While GPT-5 remains the overall top performer, the gap between models varies by task. Create and Repair tasks generally yield higher *Pass Rates* and coverage gains, suggesting that generating or fixing tests aligns well with model capabilities. Update task, in contrast, remains challenging: all models show reduced $\overline{\Delta}$MutCov, reflecting the difficulty of precise context-aware edits. Interestingly, GPT-OSS-120B achieves competitive results in Update, approaching GPT-5 in *Pass Rate* and $\overline{\Delta}$TestCov. This may reflect better alignment with structured modification patterns.

Within the test creation task, models achieve the highest $\overline{\Delta}$TestCov and $\overline{\Delta}$MutCov in the *From Scratch* scenario, where entire test suites must be synthesized. This is partly due to the fact that coverage metrics in this setting start from zero — making any successful generation yield large absolute gains. In contrast, improving already partially tested code, as in *Add New Tests* or *Recover Tests*, proves harder: the higher the initial coverage, the more challenging it becomes to find meaningful, non-redundant testing paths. Remarkably, *Add New Tests* and *Recover Tests* show relatively high *Pass Rates* but negligible coverage improvements, indicating limited semantic depth in

TABLE III
MAIN RESULTS BY LANGUAGE (ATTEMPT@3)

Model	Go			Java			Python		
	Pass Rate (%)	$\overline{\Delta}$ TestCov	$\overline{\Delta}$ MutCov	Pass Rate (%)	$\overline{\Delta}$ TestCov	$\overline{\Delta}$ MutCov	Pass Rate (%)	$\overline{\Delta}$ TestCov	$\overline{\Delta}$ MutCov
Devstral-Small	↑21.1	↑10.7	↑6.6	↑3.4	↑0.2	↓0.0	↑5.9	↑2.0	↓0.7
Qwen3 Coder 480B A35B	↑34.4	↑17.7	↑12.1	↑6.3	↑1.1	↑0.2	↑10.9	↑4.7	↑0.6
DeepSeek V3.1 671B	↑18.9	↑11.9	↑7.6	↑4.6	↑1.7	↓0.0	↑5.0	↑1.9	↓0.1
GPT-OSS-120B	**↑56.5**	**↑30.9**	**↑21.7**	**↑14.3**	**↑4.2**	**↑0.6**	**↑21.3**	**↑10.0**	**↑1.1**
Gemini 2.5 Flash	↑17.1	↑9.7	↑6.3	↑6.1	↑1.3	↓0.0	↑4.5	↑1.9	**↑0.6**
GPT-5	**↑67.8**	**↑38.8**	**↑28.1**	**↑29.3**	**↑8.1**	**↑1.9**	**↑20.4**	**↑9.2**	↑0.5

model completions. Tables with models' performance by task and scenario are provided in our repository [9].

D. Failures Analysis

Fig. 3 reveals that the dominant failure mode across all models is `execution_error`, accounting for over 60% of invalid outputs for most of the evaluated LLMs. This indicates persistent challenges in maintaining runtime-correct behavior, even when syntactic validity is achieved. Notably, models like GPT-OSS-120B and DeepSeek V3.1 671B exhibit relatively higher `pass` rates, suggesting stronger capabilities in producing semantically correct tests. In contrast, Devstral-Small and Gemini 2.5 Flash suffer from excessive execution-level failures (up to 80%).

Among the evaluated models, Qwen3 Coder 480B A35B demonstrates the highest frequency of introducing unnecessary introductory text into its output.

A notable issue observed in Go-generated tests was improper import management, specifically unused imports, along with frequent occurrences of undefined names and function references.

Python tests generated by the Devstral-Small model exhibited the broadest spectrum of exception types, encompassing ValueError, NotImplementedError, TypeError, and others.

Finally, for Java-generated tests, the most prevalent exceptions encountered were NullPointerException and IndexOutOfBoundsException.

In general, all models have errors for the same files. Only several samples were misgenerated by one or two models. It points out that all models have similar weaknesses. The Go generated tests have the highest number of errors that can be caused by the dominant amount of data in Java and Python for the models training.

Interestingly, syntax error rates remain low, implying that most models have largely mastered surface-level code formatting. Overall, setup of correct and executable test suite emerges as the principal bottleneck in achieving reliable test maintenance.

VII. CONCLUSION

This work presents TAM-Eval, a novel benchmark and evaluation framework for assessing large language models (LLMs) in automated unit test maintenance across Python, Java, and Go. The benchmark comprises 1,539 task instances, carefully curated through a multi-stage process to reflect practical software engineering scenarios, with a modular design that supports future extensions to additional languages and repositories.

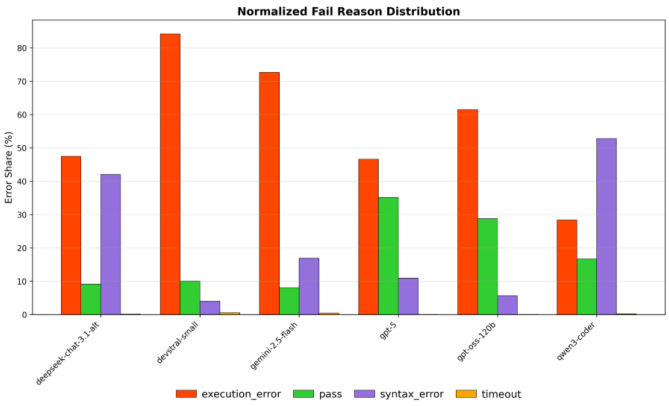

Fig. 3. Distribution of generated test suites fail reason (attempt@3).

The TAM-Eval framework employs reference-free metrics – test suite pass rates, test and mutation coverage gains – to evaluate abilities of creation, repairing, and updating unit tests, revealing significant insights into model performance.

Our baseline analysis reveals significant shortcomings in LLM performance for unit test maintenance. Although performance improves with multiple attempts underscoring the importance of iterative feedback from automated verifier systems such as compilers.

These findings highlight the potential of LLMs in test maintenance while identifying key areas for improvement, including enhanced context handling and support for higher-order testing paradigms, paving the way for future research in automated software engineering. Beyond the experiments reported here, the TAM-Eval benchmark has also been used as a validation set for training reward models [42], demonstrating its applicability for related downstream tasks.

[9] https://github.com/trndcenter/TAM-Eval

REFERENCES

[1] D. Huizinga and A. Kolawa, *Automated Defect Prevention: Best Practices in Software Management*. Wiley-IEEE Computer Society Pr, 2007.

[2] Global App Testing, "How Much Does Software Testing Cost in 2025? — globalapptesting.com," https://www.globalapptesting.com/blog/software-testing-cost, [Accessed 26-05-2025].

[3] J. Jiang, F. Wang *et al.*, "A survey on large language models for code generation," 2024. [Online]. Available: https://arxiv.org/abs/2406.00515

[4] L. Yang, C. Yang *et al.*, "On the evaluation of large language models in unit test generation," 2024. [Online]. Available: https://arxiv.org/abs/2406.18181

[5] S. S. Dvivedi, V. Vijay *et al.*, "A comparative analysis of large language models for code documentation generation," in *Proceedings of the 1st ACM International Conference on AI-Powered Software*, ser. AIware 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 65–73. [Online]. Available: https://doi.org/10.1145/3664646.3664765

[6] J. Cordeiro, S. Noei, and Y. Zou, "An empirical study on the code refactoring capability of large language models," 2024. [Online]. Available: https://arxiv.org/abs/2411.02320

[7] X. Meng, Z. Ma *et al.*, "An empirical study on llm-based agents for automated bug fixing," 2024. [Online]. Available: https://arxiv.org/abs/2411.10213

[8] W. Miller and D. L. Spooner, "Automatic generation of floating-point test data," *IEEE Trans. Softw. Eng.*, vol. 2, no. 3, p. 223–226, May 1976. [Online]. Available: https://doi.org/10.1109/TSE.1976.233818

[9] C. Pacheco and M. D. Ernst, "Randoop: feedback-directed random testing for java," in *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion*, ser. OOPSLA '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 815–816. [Online]. Available: https://doi.org/10.1145/1297846.1297902

[10] G. Fraser and A. Arcuri, "Evosuite: automatic test suite generation for object-oriented software," in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, ser. ESEC/FSE '11. New York, NY, USA: Association for Computing Machinery, 2011, p. 416–419. [Online]. Available: https://doi.org/10.1145/2025113.2025179

[11] S. Gao, C. Wang *et al.*, "The prompt alchemist: Automated llm-tailored prompt optimization for test case generation," 2025. [Online]. Available: https://arxiv.org/abs/2501.01329

[12] J. Huang, X. Chen *et al.*, "Large language models cannot self-correct reasoning yet," 2024. [Online]. Available: https://arxiv.org/abs/2310.01798

[13] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *International Conference on Learning Representations (ICLR)*, 2024.

[14] J. Wei, X. Wang *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2022.

[15] S. Yao, D. Yu *et al.*, "Tree of thoughts: deliberate problem solving with large language models," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.

[16] Z. Yuan, M. Liu *et al.*, "Evaluating and improving chatgpt for unit test generation," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024. [Online]. Available: https://doi.org/10.1145/3660783

[17] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring llm-based general bug reproduction," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 2312–2323. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00194

[18] S. Gu, Q. Zhang *et al.*, "Testart: Improving llm-based unit testing via co-evolution of automated generation and repair iteration," 2025. [Online]. Available: https://arxiv.org/abs/2408.03095

[19] Y. Chen, Z. Hu *et al.*, "Chatunitest: A framework for llm-based test generation," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, ser. FSE 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 572–576. [Online]. Available: https://doi.org/10.1145/3663529.3663801

[20] S. Gu, N. Nashid, and A. Mesbah, "Llm test generation via iterative hybrid program analysis," 2025. [Online]. Available: https://arxiv.org/abs/2503.13580

[21] M. Schäfer, S. Nadi *et al.*, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2024.

[22] N. Mündler, M. N. Müller *et al.*, "Swt-bench: Testing and validating real-world bug-fixes with code agents," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey *et al.*, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 81 857–81 887.

[23] N. Alshahwan, J. Chheda *et al.*, "Automated unit test improvement using large language models at meta," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, ser. FSE 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 185–196. [Online]. Available: https://doi.org/10.1145/3663529.3663839

[24] A. Challande, R. David, and G. Renault, "Building a commit-level dataset of real-world vulnerabilities," in *Proceedings of the Twelfth ACM Conference on Data and Application Security and Privacy*, ser. CODASPY '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 101–106. [Online]. Available: https://doi.org/10.1145/3508398.3511495

[25] E. Bogomolov, A. Eliseeva *et al.*, "Long code arena: a set of benchmarks for long-context code models," 2024. [Online]. Available: https://arxiv.org/abs/2406.11612

[26] J. Yang, K. Leret *et al.*, "Swe-smith: Scaling data for software engineering agents," 2025. [Online]. Available: https://arxiv.org/abs/2504.21798

[27] Q. Zhang, Y. Shang *et al.*, "Testbench: Evaluating class-level test case generation capability of large language models," *arXiv preprint arXiv:2409.17561*, 2024.

[28] Y. Wang, C. Xia *et al.*, "Projecttest: A project-level llm unit test generation benchmark and impact of error fixing mechanisms," 2025. [Online]. Available: https://arxiv.org/abs/2502.06556

[29] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VTF8yNQM66

[30] V. Bhargava, R. Ghosh, and D. Dutta, "Cpp-ut-bench: Can llms write complex unit tests in c++?" 2024. [Online]. Available: https://arxiv.org/abs/2412.02735

[31] J. Xu, B. Pang, J. Qu, H. Hayashi, C. Xiong, and Y. Zhou, "Clover: A test case generation benchmark with coverage, long-context, and verification," 2025. [Online]. Available: https://arxiv.org/abs/2502.08806

[32] W. Wang, C. Yang *et al.*, "TESTEVAL: Benchmarking large language models for test case generation," in *Findings of the Association for Computational Linguistics: NAACL 2025*, L. Chiruzzo, A. Ritter, and L. Wang, Eds. Albuquerque, New Mexico: Association for Computational Linguistics, Apr. 2025, pp. 3547–3562. [Online]. Available: https://aclanthology.org/2025.findings-naacl.197/

[33] B. Li, W. Wu *et al.*, "Prompting large language models to tackle the full software development lifecycle: A case study," 2024. [Online]. Available: https://arxiv.org/abs/2403.08604

[34] K. Jain, G. Synnaeve, and B. Rozière, "Testgeneval: A real world unit test generation and test completion benchmark," 2025. [Online]. Available: https://arxiv.org/abs/2410.00752

[35] A. Chervyakov, A. Kharitonov *et al.*, "Mera code: A unified framework for evaluating code generation across tasks," 2025. [Online]. Available: https://arxiv.org/abs/2507.12284

[36] Mistral AI, "DevStral: Introducing the best open-source model for coding agents." https://mistral.ai/news/devstral, 2025, accessed: 2025-05-30.

[37] A. Yang, A. Li *et al.*, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[38] DeepSeek-AI, "Deepseek-v3 technical report," 2024. [Online]. Available: https://arxiv.org/abs/2412.19437

[39] OpenAI, "Gpt-oss-120b & gpt-oss-20b model card," 2025. [Online]. Available: https://arxiv.org/abs/2508.10925

[40] G. Comanici, E. Bieber *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv e-prints*, p. arXiv:2507.06261, Jul. 2025.

[41] OpenAI, "Introducing GPT-5 in the API," https://openai.com/index/introducing-gpt-5/, 2025, accessed: 2025-10-28.

[42] E. Bruches, D. Grebenkin *et al.*, "RM -RF: Reward model for run-free unit test evaluation," in *Proceedings of the 33rd IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2026, to be published. [Online]. Available: https://arxiv.org/abs/2601.13097